Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Open course index

Side Block Position

Skip Quiz navigation

Quiz navigation

Question 1 This page

Finish review

1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

Started on	Thursday, 24 September 2026, 10:25 AM
State	Finished
Completed on	Thursday, 24 September 2026, 10:26 AM
Time taken	58 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer:(penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n, i, j;

    scanf("%d", &n);

    int a[n];

    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int n, i, j;

    scanf("%d", &n);

    int a[n];

```

```
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

Feedback

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4	1	1

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

Footer (Left)
Footer (Middle)
Footer (Middle 2)
Footer (Right)
You are logged in as Priyadharshini C 251801247-2025-AIDS (Log out)
Data retention summary

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review



0

Notifications

You have no notifications
See all
Priyadharshini C 251801247-2025-AIDS P2
Profile Grades Calendar Private files Reports
Preferences
Log out
P2

- Home
- Dashboard
- My courses
- More

Search courses Search courses
Open course index
Open block drawer

1. CS23331-DAA-2025-CSE
2. 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

Started on	Thursday, 24 September 2026, 10:26 AM
State	Finished
Completed on	Thursday, 24 September 2026, 10:28 AM
Time taken	1 min 36 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer:(penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n, i;
    scanf("%d", &n);

    int a[n];

    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    int slow = a[0];
    int fast = a[0];

    // Find meeting point
    do {
        slow = a[slow];
        fast = a[a[fast]];
    } while (slow != fast);

    // Find duplicate
    slow = a[0];

    while (slow != fast) {
        slow = a[slow];
        fast = a[fast];
    }

    printf("%d", slow);

    return 0;
}

```

```c
#include <stdio.h>

int main() {
    int n, i;
    scanf("%d", &n);

    int a[n];

    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    int slow = a[0];
    int fast = a[0];

    // Find meeting point
    do {
        slow = a[slow];
        fast = a[a[fast]];
    } while (slow != fast);

    // Find duplicate
    slow = a[0];

    while (slow != fast) {
        slow = a[slow];
        fast = a[fast];
    }

    printf("%d", slow);

    return 0;
}
```

Feedback

Input	Expected	Got
11 10 9 7 6 5 1 2 3 8 4 7	7	7
5 1 2 3 4 4	4	4
5 1 1 2 3 4	1	1

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course



You are logged in as Priyadharshini C 251801247-2025-AIDS (Log out)
Data retention summary

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review

3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

Started on	Thursday, 24 September 2026, 10:28 AM
State	Finished
Completed on	Thursday, 24 September 2026, 10:29 AM
Time taken	56 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array

2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer:(penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int T, n1, n2, i, j;
```

```

    scanf("%d", &T);

    while (T--) {
        scanf("%d", &n1);

        int a[n1];

        for (i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

        int b[n2];

        for (i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        for (i = 0; i < n1; i++) {
            for (j = 0; j < n2; j++) {
                if (a[i] == b[j]) {
                    printf("%d ", a[i]);
                    break;
                }
            }
        }

        printf("\n");
    }

    return 0;
}


```

```
#include <stdio.h>

int main() {
    int T, n1, n2, i, j;

    scanf("%d", &T);

    while (T--) {
        scanf("%d", &n1);

        int a[n1];

        for (i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

        int b[n2];

        for (i = 0; i < n2; i++)
            scanf("%d", &b[i]);

```

```
        for (i = 0; i < n1; i++) {
            for (j = 0; j < n2; j++) {
                if (a[i] == b[j]) {
                    printf("%d ", a[i]);
                    break;
                }
            }
        }

        printf("\n");
    }

    return 0;
}
```

Feedback

Input	Expected	Got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57
1 6 1 2 3 4 5 6 2 1 6	1 6	1 6

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

Started on	Thursday, 24 September 2026, 10:29 AM
State	Finished
Completed on	Thursday, 24 September 2026, 11:16 AM
Time taken	46 mins 21 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array

2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer:(penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int T, n1, n2, i, j;
```

```

    scanf("%d", &T);

    while (T--) {
        scanf("%d", &n1);

        int a[n1];
        for (i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

        int b[n2];
        for (i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        i = 0;
        j = 0;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j]) {
                i++;
            }
            else {
                j++;
            }
        }

        printf("\n");
    }

    return 0;
}


```

```
#include <stdio.h>

int main() {
    int T, n1, n2, i, j;

    scanf("%d", &T);

    while (T--) {
        scanf("%d", &n1);

        int a[n1];
        for (i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

```

```
        int b[n2];
        for (i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        i = 0;
        j = 0;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j]) {
                i++;
            }
            else {
                j++;
            }
        }

        printf("\n");
    }

    return 0;
}
```

Feedback

Input	Expected	Got
1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57
1 6 1 2 3 4 5 6 2 1 6	1 6	1 6

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

Footer (Left)
Footer (Middle)
Footer (Middle 2)
Footer (Right)

You are logged in as Priyadharshini C 251801247-2025-AIDS (Log out)
Data retention summary

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review

5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

Started on	Thursday, 24 September 2026, 11:16 AM
State	Finished
Completed on	Thursday, 24 September 2026, 11:17 AM
Time taken	1 min 22 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 1 4	

Answer:(penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int A[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &A[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        } else {
            int diff = A[j] - A[i];

            if (diff == k) {
                printf("1");
                return 0;
            } else if (diff < k) {
                j++;
            } else {
```

```
                i++;
            }
        }
    }

    printf("0");
    return 0;
}
```

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int A[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &A[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        } else {
            int diff = A[j] - A[i];

            if (diff == k) {
                printf("1");
                return 0;
            } else if (diff < k) {
                j++;
            } else {
                i++;
            }
        }
    }

    printf("0");
    return 0;
}
```

Feedback

Input	Expected	Got
3 1 3 5 4	1	1
10 1 4 6 8 12 14 15 20 21 25 1	1	1
10 1 2 3 5 11 14 16 24 28 29 0	0	0
10 0 2 3 7 13 14 15 20 24 25 10	1	1

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

undefined

File:
Line:
Stack trace:

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position

Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review



0

Notifications

You have no notifications
See all
Priyadharshini C 251801247-2025-AIDS P2
Profile Grades Calendar Private files Reports
Preferences
Log out
P2

- Home
- Dashboard
- My courses
- More

Search courses Search courses
Open course index
Open block drawer

1. CS23331-DAA-2025-CSE
2. 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Started on	Thursday, 24 September 2026, 11:25 AM
State	Finished
Completed on	Thursday, 24 September 2026, 11:31 AM
Time taken	5 mins 59 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 1 4	

Answer:(penalty regime: 0 %)

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int A[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &A[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        }
        else if (A[j] - A[i] == k) {
            printf("1");
            return 0;
        }
        else if (A[j] - A[i] < k) {
            j++;
        }
```

```
        else {
            i++;
        }
    }

    printf("0");

    return 0;
}
```

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int A[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &A[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        }
        else if (A[j] - A[i] == k) {
            printf("1");
            return 0;
        }
        else if (A[j] - A[i] < k) {
            j++;
        }
        else {
            i++;
        }
    }

    printf("0");

    return 0;
}
```

Feedback

Input	Expected	Got
3 1 3 5 4	1	1
10 1 4 6 8 12 14 15 20 21 25 1	1	1
10 1 2 3 5 11 14 16 24 28 29 0	0	0
10 0 2 3 7 13 14 15 20 24 25 10	1	1

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course